IT305 Lab: Socket Programming Project

Complete Windows & WSL2 Setup, Build, and Testing Guide

Course: IT305 Computer Networks Lab (DAIICT) | **Merged Groups:** Group 103 & Group 106 | **Target:** Linux / WSL2

This project implements a **Fault-Tolerant Topic-Based File Distribution Service** in modern C++17 leveraging low-level POSIX sockets, multi-threading, Linux non-blocking epoll, zero-copy sendfile(2), and atomic checkpoint durability. Because Windows does not natively provide POSIX socket headers or epoll, the system runs seamlessly on Windows through **Windows Subsystem for Linux (WSL2)**. This guide walks any user through environment preparation, dependency installation, building all 4 tiers, and running verification suites.

1. One-Time Environment Setup (Windows WSL2)

If you do not have WSL2 installed, open **PowerShell as Administrator** and run:

```
wsl --install -d Ubuntu
```

Restart Windows if prompted. Launch **Ubuntu** from the Start menu and set up your username/password.

2. Install Required Tools Inside WSL

Inside your Ubuntu terminal, install the standard C++17 build toolchain and benchmark utilities:

```
sudo apt update
sudo apt install -y build-essential g++ make python3 python3-pandas python3-matplotlib dos2unix
```

3. Navigate to the Repository in WSL

Windows drives are automatically mounted under /mnt/c/:

```
# Replace  with your actual Windows username:
cd "/mnt/c/Users//Downloads/College_Project/CN_Project/GH_Project"
ls -la
```

Verify you see directories: Part1, Part2_Case1, Part2_Case2, Part2_Case2_Enhanced, Dataset, Results, and test.

4. Run Standalone Unit Tests (Fast Verification)

Run the integrated test harness containing **338 assertions** with zero external dependencies:

```
cd test
make test
cd ..
```

Test Suite	Assertions	Focus Area
`test_protocol`	~108	Wire framing, opcode encoding, big-endian network byte order, payload safety
`test_mapper`	~79	Topic directory scanning, filename sorting, path traversal guards
`test_fault`	~60	Probabilistic chunk-boundary drop rate, seed reproducibility
`test_session_ckpt`	15 / 15	128-bit UUID generation, fsync durability, 7-step resume validation
`test_enhanced`	76 / 76	4-stream TCP parallelism, sendfile zero-copy, bitmask recovery

5. Build All Four Project Tiers

Each folder compiles independently with strict flags (-O2 -Wall -Wextra -Werror -pthread -std=c++17):

```
cd Part1 && make clean && make && cd ..
cd Part2_Case1 && make clean && make && cd ..
cd Part2_Case2 && make clean && make && cd ..
cd Part2_Case2_Enhanced && make clean && make && cd ..
```

6. Interactive Execution Guide (Two Terminal Setup)

Open **two separate WSL terminal windows side-by-side** to observe real-time network transfers:

Scenario A: Part 1 - Reliable Transfer (Multi-Threaded / Single-Threaded)

- **Terminal 1 (Server):**

```
cd Part1
./server 9000 ../Dataset/raw-img
```

- **Terminal 2 (Client):**

```
cd Part1
./client 127.0.0.1 9000 cane /tmp/client_p1
# Verify bit-for-bit file integrity (MD5 / diff):
diff -r ../Dataset/raw-img/cane /tmp/client_p1/cane && echo 'MATCH: 100% IDENTICAL'
```

Scenario B: Part 2 Case 1 - Stateless Retry under Fault Injection

- **Terminal 1 (Server drops connection on ~30% of chunks):**

```
cd Part2_Case1
./server 9001 ../Dataset/raw-img 0.3
```

- **Terminal 2 (Client retries from byte 0 up to 5 times):**

```
cd Part2_Case1
./client 127.0.0.1 9001 cane /tmp/client_c1
```

Scenario C: Part 2 Case 2 - Stateful Checkpointed Resume

- **Terminal 1 (Server maintains session registry & checkpoint state):**

```
cd Part2_Case2
./server 9002 ../Dataset/raw-img 0.3
```

- **Terminal 2 (Client seamlessly resumes from last durable .ckpt on drop):**

```
cd Part2_Case2
./client 127.0.0.1 9002 cane /tmp/client_c2
```

Scenario D: Part 2 Case 2 Enhanced - 4 Parallel TCP Streams + epoll + sendfile

- **Terminal 1 (High-performance event-driven epoll server):**

```
cd Part2_Case2_Enhanced
./server 9003 ../Dataset/raw-img 0.3
```

- **Terminal 2 (Client splits chunks across 4 streams with bitmask recovery):**

```
cd Part2_Case2_Enhanced
./client 127.0.0.1 9003 cane /tmp/client_enh
```

7. Automated Benchmarking & Visualization

Run the automated orchestration script to execute all matrix permutations across N={1,2,4,8,16,32} and Pf={0.0,0.1,0.3,0.5}:

```
cd Results
chmod +x run_benchmarks.sh plot_results.py
dos2unix run_benchmarks.sh plot_results.py 2>/dev/null
bash run_benchmarks.sh
python3 plot_results.py
```

This will automatically generate the 3 required report figures:

- Part1_SingleVsMulti_throughput.png - Aggregate goodput scaling vs. client count N
- Part2_AllCases_throughput.png - Client goodput vs. failure probability Pf (N=8)
- Part2_AllCases_overhead.png - Retransmission overhead % vs. failure probability Pf (N=8)

8. Troubleshooting & Common Pitfalls

Symptom / Error	Root Cause	Quick Resolution
`Address already in use`	Previous server still in TIME_WAIT	Run killall server or wait 30 seconds for socket reuse.
`\r: command not found`	Windows CRLF line endings in scripts	Run dos2unix run_benchmarks.sh in WSL.
`epoll_create1 failed`	WSL1 emulation layer used	Upgrade WSL: in PowerShell, run wsl --set-version Ubuntu 2.
Permission denied on Dataset	Missing read execution permissions	Run chmod -R 755 Dataset/ in WSL.